

Vježba 2: Forma s više inputa

Naziv projekta

08_React/06_Practice_MultipleInputs/

Cilj vježbe

U ovoj vježbi učenici će napraviti malo veću formu koja koristi više inputa, ali samo jedan state objekt i jedan `handleChange` handler.

Vježba pokriva:

- kontrolirane tekstualne inpute
 - `textarea`
 - `select`
 - `checkbox`
 - `onSubmit`
 - `event.preventDefault()`
 - jedan state objekt za cijelu formu
 - jedan `handleChange` handler
 - computed property name sintaksu: `[name]`
 - uvjetno prikazivanje nakon slanja forme
-

Scenarij

Zamislite da izrađujete formu za prijavu na kratki React workshop.

Korisnik treba unijeti:

First name

Last name

Email

Experience level

Motivation message

Accept terms

Kada korisnik pošalje formu, stranica se ne smije ponovno učitati. Umjesto toga, aplikacija treba prikazati sažetak prijave.

Zadatak

1. Kreirajte novi React projekt

U terminalu pokrenite:

```
npm create vite@latest workshop-registration-form -- --template react
```

Uđite u projekt:

```
cd workshop-registration-form
```

Instalirajte pakete:

```
npm install
```

Pokrenite aplikaciju:

```
npm run dev
```

2. Očistite početni projekt

U `src/App.jsx` pripremite osnovnu strukturu:

```
import { useState } from "react";
import "./App.css";

function App() {
  return (
    <main className="app">
      <h1>React Workshop Registration</h1>
    </main>
  );
}

export default App;
```

U `src/App.css` obrišite postojeći CSS.

3. Dodajte jedan state objekt za cijelu formu

Unutar `App` komponente dodajte:

```
const [formData, setFormData] = useState({
  firstName: "",
  lastName: "",
  email: "",
  experience: "beginner",
  motivation: "",
  acceptTerms: false
});
```

Ovdje svi podaci forme žive u jednom objektu.

To znači:

```
formData.firstName
formData.lastName
formData.email
formData.experience
formData.motivation
formData.acceptTerms
```

4. Dodajte state za slanje forme

Dodajte još jedan state koji prati je li forma poslana:

```
const [submitted, setSubmitted] = useState(false);
```

Na početku je **false**, jer korisnik još nije poslao formu.

5. Napravite jedan **handleChange** handler

Dodajte funkciju:

```
function handleChange(event) {
  const { name, value, type, checked } = event.target;

  setFormData(prev => ({
    ...prev,
    [name]: type === "checkbox" ? checked : value
  }));
}
```

Ova funkcija radi za sve fieldove.

Za obične inpute, **textarea** i **select**, koristi se **value**.

Za checkbox se koristi `checked`.

Najvažniji dio je:

```
[name]: type === "checkbox" ? checked : value
```

`name` atribut inputa odlučuje koje se svojstvo u state objektu ažurira.

6. Napravite `handleSubmit` funkciju

Dodajte funkciju:

```
function handleSubmit(event) {  
  event.preventDefault();  
  setSubmitted(true);  
}
```

`event.preventDefault()` sprječava ponovno učitavanje stranice.

Bez toga bi preglednik poslao formu na klasičan HTML način i React state bi se izgubio.

7. Dodajte formu

Forma treba imati `onSubmit`:

```
<form onSubmit={handleSubmit} className="form">
```

Dodajte fieldove jedan po jedan.

8. Dodajte input za ime

```
<div className="form-group">  
  <label htmlFor="firstName">First name:</label>  
  <input  
    id="firstName"  
    name="firstName"  
    type="text"  
    value={formData.firstName}  
    onChange={handleChange}  
  />  
</div>
```

Obratite pažnju na `name="firstName"`.

To mora odgovarati nazivu svojstva u state objektu:

firstName: ""

9. Dodajte input za prezime

```
<div className="form-group">
  <label htmlFor="lastName">Last name:</label>
  <input
    id="lastName"
    name="lastName"
    type="text"
    value={formData.lastName}
    onChange={handleChange}
  />
</div>
```

10. Dodajte input za email

```
<div className="form-group">
  <label htmlFor="email">Email:</label>
  <input
    id="email"
    name="email"
    type="email"
    value={formData.email}
    onChange={handleChange}
  />
</div>
```

11. Dodajte `select` za iskustvo

Korisnik treba odabrati razinu iskustva:

Beginner
Intermediate
Advanced

Dodajte:

```
<div className="form-group">
  <label htmlFor="experience">Experience level:</label>
```

```
<select
  id="experience"
  name="experience"
  value={formData.experience}
  onChange={handleChange}
>
  <option value="beginner">Beginner</option>
  <option value="intermediate">Intermediate</option>
  <option value="advanced">Advanced</option>
</select>
</div>
```

U Reactu `select` koristi `value` na `select` elementu, a ne `selected` na `option` elementu.

12. Dodajte `textarea` za motivaciju

```
<div className="form-group">
  <label htmlFor="motivation">Why do you want to join?</label>
  <textarea
    id="motivation"
    name="motivation"
    value={formData.motivation}
    onChange={handleChange}
    rows={4}
  />
</div>
```

U Reactu `textarea` koristi `value`, kao i obični input.

13. Dodajte checkbox za uvjete korištenja

```
<div className="checkbox-group">
  <label>
    <input
      name="acceptTerms"
      type="checkbox"
      checked={formData.acceptTerms}
      onChange={handleChange}
    />
    {" "}I accept the terms and conditions
  </label>
</div>
```

Kod checkboxa ne koristimo `value`, nego `checked`.

Također, u `handleChange` funkciji ne čitamo samo `event.target.value`, nego razlikujemo `value` i `checked`.

14. Dodajte submit gumb

```
<button type="submit">
  Register
</button>
```

Gumb mora imati `type="submit"` kako bi pokrenuo `onSubmit` forme.

15. Prikažite potvrdu nakon slanja

Ako je `submitted` vrijednost `true`, prikažite sažetak prijave umjesto forme.

Prije glavnog `return` dijela dodajte:

```
if (submitted) {
  return (
    <main className="app">
      <section className="summary-card">
        <h1>Registration Summary</h1>

        <p><strong>Name:</strong> {formData.firstName} {formData.lastName}</p>
        <p><strong>Email:</strong> {formData.email}</p>
        <p><strong>Experience:</strong> {formData.experience}</p>
        <p><strong>Motivation:</strong> {formData.motivation}</p>
        <p>
          <strong>Terms accepted:</strong>{" "}
          {formData.acceptTerms ? "Yes" : "No"}
        </p>

        <button onClick={() => setSubmitted(false)}>
          Edit Form
        </button>
      </section>
    </main>
  );
}
```

Ovo je primjer uvjetnog renderiranja pomoću ranog `return`.

16. Dodajte reset forme

Dodajte funkciju:

```
function resetForm() {  
  setFormData({  
    firstName: "",  
    lastName: "",  
    email: "",  
    experience: "beginner",  
    motivation: "",  
    acceptTerms: false  
  });  
  
  setSubmitted(false);  
}
```

Zatim u summary prikazu dodajte još jedan gumb:

```
<button onClick={resetForm}>  
  Start Over  
</button>
```

Očekivano ponašanje aplikacije

Kada korisnik otvori aplikaciju, vidi formu za prijavu.

Korisnik može upisati ime, prezime, email, odabrati razinu iskustva, napisati motivaciju i označiti checkbox.

Kada klikne **Register**, stranica se ne učitava ponovno.

Umjesto forme prikazuje se summary s unesenim podacima.

Klik na **Edit Form** vraća korisnika na formu s podacima koji su i dalje sačuvani.

Klik na **Start Over** resetira formu i vraća korisnika na početno stanje.

Puno rješenje

src/App.jsx

```
import { useState } from "react";
import "./App.css";

function App() {
  const [formData, setFormData] = useState({
    firstName: "",
    lastName: "",
    email: "",
    experience: "beginner",
    motivation: "",
    acceptTerms: false
  });

  const [submitted, setSubmitted] = useState(false);

  function handleChange(event) {
    const { name, value, type, checked } = event.target;

    setFormData(prev => ({
      ...prev,
      [name]: type === "checkbox" ? checked : value
    }));
  }

  function handleSubmit(event) {
    event.preventDefault();
    setSubmitted(true);
  }

  function resetForm() {
    setFormData({
      firstName: "",
      lastName: "",
      email: "",
      experience: "beginner",
      motivation: "",
      acceptTerms: false
    });

    setSubmitted(false);
  }

  if (submitted) {
    return (
      <main className="app">
        <section className="summary-card">
```

```

    <h1>Registration Summary</h1>

    <p>
      <strong>Name:</strong> {formData.firstName} {formData.lastName}
    </p>

    <p>
      <strong>Email:</strong> {formData.email}
    </p>

    <p>
      <strong>Experience:</strong> {formData.experience}
    </p>

    <p>
      <strong>Motivation:</strong> {formData.motivation}
    </p>

    <p>
      <strong>Terms accepted:</strong>{" "}
      {formData.acceptTerms ? "Yes" : "No"}
    </p>

    <div className="button-row">
      <button onClick={() => setSubmitted(false)}>
        Edit Form
      </button>

      <button onClick={resetForm}>
        Start Over
      </button>
    </div>
  </section>
</main>
);
}

return (
  <main className="app">
    <h1>React Workshop Registration</h1>

    <form onSubmit={handleSubmit} className="form">
      <div className="form-group">
        <label htmlFor="firstName">First name:</label>
        <input
          id="firstName"
          name="firstName"
          type="text"

```

```
        value={formData.firstName}
        onChange={handleChange}
      />
    </div>

    <div className="form-group">
      <label htmlFor="lastName">Last name:</label>
      <input
        id="lastName"
        name="lastName"
        type="text"
        value={formData.lastName}
        onChange={handleChange}
      />
    </div>

    <div className="form-group">
      <label htmlFor="email">Email:</label>
      <input
        id="email"
        name="email"
        type="email"
        value={formData.email}
        onChange={handleChange}
      />
    </div>

    <div className="form-group">
      <label htmlFor="experience">Experience level:</label>
      <select
        id="experience"
        name="experience"
        value={formData.experience}
        onChange={handleChange}
      >
        <option value="beginner">Beginner</option>
        <option value="intermediate">Intermediate</option>
        <option value="advanced">Advanced</option>
      </select>
    </div>

    <div className="form-group">
      <label htmlFor="motivation">Why do you want to join?</label>
      <textarea
        id="motivation"
        name="motivation"
        value={formData.motivation}
        onChange={handleChange}
      />
    </div>
```

```

        rows={4}
      />
    </div>

    <div className="checkbox-group">
      <label>
        <input
          name="acceptTerms"
          type="checkbox"
          checked={formData.acceptTerms}
          onChange={handleChange}
        />
        {" "}I accept the terms and conditions
      </label>
    </div>

    <button type="submit">
      Register
    </button>
  </form>
</main>
);
}

```

```
export default App;
```

src/App.css

```

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #eef2f7;
}

```

```

.app {
  max-width: 560px;
  margin: 50px auto;
  padding: 32px;
  background-color: white;
  border: 1px solid #d8dee9;
  border-radius: 16px;
}

```

```

h1 {
  margin-top: 0;
  text-align: center;
}

```

```
}
```

```
.form {  
  margin-top: 24px;  
}
```

```
.form-group {  
  margin-bottom: 18px;  
}
```

```
label {  
  display: block;  
  margin-bottom: 8px;  
  font-weight: bold;  
}
```

```
input,  
select,  
textarea {  
  width: 100%;  
  box-sizing: border-box;  
  padding: 10px;  
  border: 1px solid #ccd3df;  
  border-radius: 8px;  
  font-size: 16px;  
  font-family: inherit;  
}
```

```
textarea {  
  resize: vertical;  
}
```

```
.checkbox-group {  
  margin-bottom: 20px;  
}
```

```
.checkbox-group label {  
  display: flex;  
  align-items: center;  
  gap: 8px;  
  font-weight: normal;  
}
```

```
.checkbox-group input {  
  width: auto;  
}
```

```
button {
```

```
padding: 10px 16px;
border: 1px solid #ccd3df;
border-radius: 8px;
background-color: #f7f9fc;
cursor: pointer;
font-size: 15px;
}
```

```
button:hover {
  background-color: #e9eef6;
}
```

```
.summary-card p {
  line-height: 1.6;
}
```

```
.button-row {
  display: flex;
  gap: 12px;
  margin-top: 24px;
}
```

Dodatni izazovi

Izazov 1: Onemogućite submit dok uvjeti nisu prihvaćeni

Gumb **Register** treba biti disabled ako **acceptTerms** nije označen.

```
<button type="submit" disabled={!formData.acceptTerms}>
  Register
</button>
```

Zatim dodajte CSS:

```
button:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}
```

Izazov 2: Prikažite live preview

Ispod forme dodajte malu karticu koja u stvarnom vremenu prikazuje unesene podatke.

Na primjer:

```
<section className="preview-card">
  <h2>Live Preview</h2>
  <p>{formData.firstName} {formData.lastName}</p>
  <p>{formData.email}</p>
  <p>{formData.experience}</p>
</section>
```

Ovo dodatno pokazuje da su vrijednosti stalno dostupne u React stateu.

Izazov 3: Dodajte novo polje

Dodajte novo polje:

Preferred schedule

Kao `select` s opcijama:

Morning
Afternoon
Evening

Učenici trebaju:

1. dodati novo svojstvo u `formData`
2. dodati novi `select`
3. postaviti odgovarajući `name`
4. prikazati vrijednost u summaryju

Ovdje se vidi prednost jednog `handleChange` handlera: nije potrebno pisati novu funkciju za novo polje.